Práctica de procesadores de lenguajes

Definición del lenguaje C-Horizon
Curso 2023–2024

Autor

Rafael Moreno Portilla

Antonio Parrilla Sánchez

Doble grado en Ingeniería Informática y Matemáticas

Facultad de Informática

Universidad Complutense de Madrid

Índice

1. Identificadores y ámbitos de definición en C-Horizon **1**
1.1. Declaración de variables 1
1.2. Bloques anidados 2
1.3. Módulos y cláusulas de importación 2
1.4. Funciones 2

2. Tipos y operadores en C-Horizon **3**
2.1. Integer 3
2.2. Boolean 3
2.3. Void 3
2.4. Array 3
2.5. Struct 4
2.6. Class 4
2.7. Punteros 5
2.8. Funcional 6
2.9. Operadores aritméticos (para enteros) 7
2.10. Operadores lógicos (para booleanos) 8
2.11. Operadores de comparación y asignación 8

3. Conjunto de instrucciones de C-Horizon **11**
3.1. Asignación 11
3.2. ; 11
3.3. While 12
3.4. For 12
3.5. If, elsif y else 12
3.6. Switch 12
3.7. Entrada y salida 12

4. Gestión de errores en C-Horizon **15**
4.1. Errores léxicos 15
4.2. Errores sintácticos 16
4.3. Errores de vinculación 17

4.3.1. Identificador ya declarado 18
4.3.2. Identificador no declarado 19
4.3.3. Tipo nuevo ya definido 19
4.3.4. Tipo nuevo no definido 20
4.3.5. Error en el número de argumentos 20

Capítulo 1

Identificadores y ámbitos de definición en C-Horizon

1.1. Declaración de variables

Para declarar una variable, se realiza de una forma muy similar a C++. Al declarar, generalmente se nombra el tipo de la variable y posteriormente el nombre de la misma. Hablemos ahora de la declaración de variables puntero, array o funcional. En el caso de los punteros, para cada tipo T podemos crear el tipo puntero a T denotado como $T*$. De la misma forma, para cada tipo T y número natural n, tenemos el tipo array de tipo T y tamaño n, denotado por $T[n]$. Finalmente, para cada tipo T y cada tupla ordenada de tipos, $Targ_1, \ldots, Targ_m$, tenemos un tipo funcional de funciones que devuelven un valor de tipo T, tienen m parámetros y el i-ésimo argumento es de tipo $Targ_i$. Denotamos este tipo funcional por $T(Targ_1, \ldots, Targ_m)$.

El nombre de una variable no puede ser cualquier cadena de caracteres, solo permitimos palabras cuyo primer carácter sea alguno de $["a" - "z"] \mid ["A" - "z"]$ y el resto de caracteres sean de $["0" - "9"] \mid ["a" - "z"] \mid ["A" - "z"] \mid "_"$. Además, permitiremos declarar una variable a la misma vez que la instanciamos con un valor.

```
int  X := 2;//Declaración y asignación a variable de tipo entero
bool  a := True;//Declaración de variable de tipo booleano
int*  p := null;//Declaración y asignación a variable de tipo entero
//Declaración y asignación a un array de tipo booleano
bool[3]  Y := [True, True, False] ;
//Declaración de un tipo funcional
int (int ) f(a) {
 return  a;
};
//Los tipos funcionales pueden devolver el tipo especial void
void (int ) f(a) {};
```

1.2. Bloques anidados

Cada par { } colocado correctamente marca un ámbito en nuestro lenguaje y recíprocamente, cada ámbito es marcado mediante este par, excepto por el ámbito total. Cada par { } debe ser colocado tras un contexto que lo permita, como tras un if + condición o tras la declaración de una función con sus argumentos. Cada ámbito puede tener variables locales, las cuales se pierden al abandonar dicho ámbito, y una tabla de simbolos propia. Además, como es razonable, se permite el anidamiento de ámbitos, es decir, subámbitos de un ámbito más grande, el superámbito, definidos mediante el anidamiento de { } o si el superámbito es el total, por incluir { }. Por tanto, los símbolos del subámbito son un superconjunto de los símbolos del superámbito. Además, se permite la ocultación de variables del superámbito al definir de nuevo nuevas variables con ese nombre en el subámbito.

1.3. Módulos y cláusulas de importación

Nuestro lenguaje permite el uso de módulos externos, que son meramente archivos de código. Para usarlos, hacemos uso de la palabra reservada `import` ñombre_archivo'. Al compilar, cada módulo es enlazado y el resultado del compilado es el programa generado por nuestro código donde en cada `import` es como si estuviera copiado el módulo entero. Por ello, es importante que los conjuntos de símbolos de todos los módulos sean disjuntos dos a dos para que las importaciones sean consistentes.

1.4. Funciones

Las funciones se declaran con un tipo funcional, un identifcador, una lista de argumentos y el cuerpo de la función (un ámbito acabado en ;).

Cada programa tiene una función especial de nombre main con tipos de retorno y argumentos sin definir hasta que la creemos, ya que es solo un símbolo especial.

Por defecto, el paso de argumentos a función es por referencia y será por valor si se indica lo contrario con el operador `$`. No obstante, este operador es un azúcar sintáctico, el tipo no cambia ya que lo único que nos indica es que debemos copiar cada argumento pasado por valor en una variable nueva auxiliar y operar sobre esa variable nueva como si fuera la antigua. No obstante, no todos los tipos permiten el paso por valor tal como es cualquier tipo funcional. También admitimos la sobrecarga de funciones si tienen parámetros diferentes, en tamaño o en tipo salvo main claramente.

Capítulo 2

Tipos y operadores en C-Horizon

En esta sección se especifican los tipos y operadores que vamos a usar. En caso de empate de prioridades, siempre asociaremos por la izquierda.

2.1. Integer

Corresponde con el *signed int* de C++. Es decir, almacena números enteros representados en sistema decimal. Tienen un espacio reservado de 32 bits, por lo que su rango de valores es [-2147483648, 2147483647]. La palabra reservada para este tipo es `int`.

2.2. Boolean

Corresponde con el *bool* de C++. Almacenará valores verdaderos (True) o falsos (False). Se realizará conversión automática de tipo `int` siendo 0 el entero que represente falso y cualquier otro entero, verdadero. La palabra reservada para este tipo es `bool`.

2.3. Void

Es el tipo vacío. Lo usaremos cuando alguna función o método no devuelva ningún valor. Su palabra reservada es `void` .

2.4. Array

Corresponde con el array de C++. Para declararlo escribiremos el tipo seguido de $[n]$ donde n es un número natural fijo que representa el tamaño. Se accederá a los elementos de la misma manera, usando el operador []. En los ejemplos siguientes vienen detallados los distintos usos:

```
1 int[1]  a; //Declaración de un array de un solo elemento
2 int[5]  b; //Declaración de un array de cinco elementos
3 int[3]  c := [2, 3, 4]; //Declaración de un array inicializado
4 int[3][2]  d := [[2,1], [3,3], [4,4]]; //Array bidimensional (array de tipo array)
5 a[0] := a[0]+a[0]; //Acceso a contenido sin inizializar (basura)
6 b[0] := 2; //Se le asigna un 2 a la primera posición del array b
7 c[2] := 6; //Se sustituye el 4 por un 6
8 d[5] := 7; //Error por mal tipado
9 d[1] := [2 , 2] ; //Se le asigna el array [2,2] a la segunda posición de d
```

Operador []

Dado un tipo array $t[n]$, el operador `[]` asigna a cada objeto array y cada entero en rango el elemento que tiene en esa posición. Además, hemos visto que también se usa como functor parcial para definir nuevos tipos o para instanciar arrays.
Operador binario.
Tipado: `[]` :: $(t[n], 1, \ldots, n-1) \rightarrow t$.
Prioridad: 12.
Asociatividad: izquierda.

2.5. Struct

Utilizaremos la palabra reservada `struct` para declararar un registro y para instanciarlo. Se realizará de manera semejante a C++. Los registros tendrán solo atributos (como es de esperar). Se accederá a los atributos con el operador "."(sin las dobles comillas). Este es un ejemplo:

```
1 struct  tEstudiante {
2   int  edad;
3   int  id;
4 };
5 int  main () {
6   struct  tEstudiante antonio := {22, 67532};
7   antonio.edad := antonio.edad + 1; //La edad se ha incrementado en 1
8 }
```

2.6. Class

Utilizaremos la palabra reservada `class` para declararar una clase. Se realizará de manera semejante a C++. Las clases tendrán atributos y métodos (atributos funcionales), todos ellos públicos. Deberán tener definido un método especial, el constructor, que debe ser único en principio. Se accederá a los otros métodos y a

los atributos con el operador "."(sin las dobles comillas) seguido del nombre del método o atributo excepto para el constructor en el que se accede solo mencionando la clase, y dependiendo de si tiene argumentos o no, se interpreta una u otra cosa. Para referenciar el constructor dentro de la clase, se usa la palabra reservada `this`.

Este es un ejemplo:

```
class Estudiante {
  Estudiante(int , int) this(e , i){
    edad := e;
    id := i;
  } ;   int  edad;
  int  id;
};
int () main() {
  Estudiante antonio := Estudiante(22, 67532);
  antonio.incr_edad(); //La edad se ha incrementado en 1
  return 0;
} ;
```

Operador .

Dado un tipo clase o struct t con conjunto de atributos/métodos A_t, el operador . asigna a cada objeto struct/clase y cada atributo/método de ese struct/clase el valor de ese campo en esa instancia. Si B_t es el conjunto de la unión del tipo de los atributos/métodos de t tenemos que
Operador binario infijo.
Tipado: . :: $(t, A_t) \to B_t$.
Prioridad: 12.
Asociatividad: izquierda.

2.7. Punteros

Para declarar un puntero a una variable, simplemente usaremos el operador *al lado del tipo. Para acceder al valor del puntero, también usaremos este operador. Cuando se declara un puntero sin instanciarlo toma el valor nullptr como en C++.

```
int * p; //Se declara un puntero
p* := 5; //Se asigna valor 5
int  num := p*; //Se accede al valor del puntero
```

Operador * (punteros)

Este símbolo, a parte de su uso en la multiplicación, nos vale para relacionar un tipo y los punteros a ese tipo pero como operador de punteros, nos sirve también para saber el valor que tiene cada puntero.
Operador unario postfijo.
Tipado: *:: t* → t.
Prioridad 11.
Asociatividad: asociativo.

2.8. Funcional

Como dijimos antes, hay tipos funcionales que están determinados por el tipo de retorno del argumento y los tipos de la tupla ordenada de argumentos. Para saber el valor para determinados argumentos de una función, se usa el operador (). Dentro del ámbito de una función, podemos usar la palabra reservada `return` para devolver un valor. Si el tipo de retorno no es void, el ámbito de la función debe ser una propiedad que llamaremos retornable. Un ámbito se dice retornable si contiene un return fuera de todo subámbito suyo o todos sus subámbitos son retornables. No permitimos prototipos de funciones, siempre se debe definir el cuerpo de la función. Mostramos ejemplos:

```
1  int(int, int) suma(a,b) {
2    return a + b;
3  } ;
4  suma(0, 1); //Invocamos a la función suma
5  int(int, int) division_entera(a,b) {
6    if  b =/= 0
7      return  a;
8    }
9    return  0 ;
10 } ;
```

Operador ()

Dado un tipo funcional $t(ta_1, \ldots, ta_n)$, el operador `()` asigna a cada función y argumentos un valor de retorno, es decir, si f es una función de tipo $t(ta_1, \ldots, ta_n)$ y a_i es de tipo ta_i para $i \in \{1, \ldots, n\}$, tenemos que `()` tiene aridad $n+1$ en este caso y $()(f, a_1, \ldots, a_n) = f(a_1, \ldots, a_n)$. Además, hemos visto que también nos permite definir nuevos tipos funcionales aunque no como operador en sí.
Operador binario viendo los argumentos como uno solo.
Tipado: `()` :: $(t(ta_1, \ldots, ta_n), ta_1, \ldots, ta_1) \to t$.
Prioridad: 12.
Asociatividad: izquierda.

Paso por valor

Indica que el argumento de la función se pasa por valor. Es azúcar sintáctico, realmente solo indica que debemos realizar una copia del argumento dentro del ámbito de la función y usarlo como nuevo argumento. Su pseudo-operador será `$` ya que al ser solo azúcar sintáctico, no es un operador como tal.
Tipado: `$`:: t → t.
Prioridad 1.
Asociatividad: no asociativo.

```
1 //Se declara una función con dos parámetros enteros, uno por valor y otro
    por referencia (por defecto es referencia)
2 int ($ int , int) sumaNum(a, b) {
3   return a + b;
4 } ;
```

2.9. Operadores aritméticos (para enteros)

Suma

Suma de dos números enteros. Se utilizará el operador `+`.
Operador binario infijo.
Tipado: `+`:: (`int`, `int`) → `int`.
Prioridad: 7.
Asociatividad: izquierda.

Resta

Resta de dos números enteros. Se utilizará el operador `-`.
Operador binario infijo.
Tipado: `-`:: (`int`, `int`) → `int`.
Prioridad: 7.
Asociatividad: izquierda.

Opuesto de un numero entero. Se utilizará el operador `-`.
Operador unario prefijo.
Tipado: `-`:: `int`→ `int`.
Prioridad: 11.
Asociatividad: asociativo.

Multiplicación

Multiplicación de dos números enteros. Se utilizará el operador `*`.
Operador binario infijo.
Tipado: `*`:: (`int`, `int`) → `int`.

Prioridad: 8.
Asociatividad: izquierda.

División

División entera de dos números enteros. Se truncará el resultado obtenido en caso de ser fraccionario. El operador utilizado será `/`.
Operador binario infijo.
Tipado: `/`:: (`int`, `int`) → `int`.
Prioridad: 8.
Asociatividad: izquierda.

2.10. Operadores lógicos (para booleanos)

Conjunción

Conjunción lógica de dos booleanos. Su operador infijo será el `&&`.
Operador binario infijo.
Tipado: `&&`:: (`bool`, `bool`) → `bool`.
Prioridad: 4.
Asociatividad: izquierda.

Disyunción

Disyunción lógica de dos booleanos. Su operador infijo será el `||`.
Operador binario infijo.
Tipado: `||`:: (`bool`, `bool`) → `bool`.
Prioridad: 3.
Asociatividad: izquierda.

Negación

Negación lógica de un booleano. Su operador prefijo será el `not`.
Operador unario prefijo.
Tipado: `not`:: `bool`→ `bool`.
Prioridad: 5.
Asociatividad: asociativo.

2.11. Operadores de comparación y asignación

Igual que

Compara dos valores de un mismo tipo y devuelve True si tienen el mismo valor, en caso contrario devuelve False.
Operador binario infijo.

Tipado: ==:: (t, t) [1] → bool.
Prioridad 3.
Asociatividad: no asociativo.

Distinto que

Equivale a not(a ==b).
Operador binario infijo.
Tipado: =/= :: (t, t) → bool.
Prioridad 6.
Asociatividad: no asociativo.

Menor que

Compara dos valores de un mismo tipo y devuelve True si el primero es menor que el segundo, en caso contrario devuelve False [2]. Su operador infijo es <.
Operador binario infijo.
Tipado: <:: (t, t) → bool.
Prioridad: 6.
Asociatividad: no asociativo.

Mayor que

Compara dos valores y devuelve True si el primero es mayor que el segundo, en caso contrario devuelve False. Su operador infijo es >.
Operador binario infijo.
Tipado: >:: (t, t) → bool.
Prioridad: 6.
Asociatividad: no asociativo.

Menor o igual que

Equivale a not(a <b).
Operador binario infijo.
Tipado: <= :: (t, t) → bool.
Prioridad: 6.
Asociatividad: no asociativo.

Mayor o igual que

Equivale a not(a >b).
Operador binario infijo.
Tipado: >= :: (t, t) → bool.

[1] Usaremos "t"para designar un tipo cualquiera
[2] Consideramos que False es menor que True

Prioridad: 6.
Asociatividad: no asociativo.

Asignación

Asigna un valor a una variable. El tipo del valor asignado debe corresponder con el tipo de la variable. Su operador infijo será ":="(sin las dobles comillas).
Operador binario infijo.
Prioridad: 1.
Asociatividad: no asociativo.

```
1 int num := 5; //Asignación válida
2 int num2 := True; //Asignación no válida por fallo de tipos
```

Capítulo 3

Conjunto de instrucciones de C-Horizon

3.1. Asignación

Nuestro primer tipo de instrucciones son las de asignación. Como hemos visto a lo largo del documento, hay una sintaxis variada para ello dependiendo del tipo de la variable. Se muestran ejemplos de todo el documento.

```
int  X := 2;
int*  p := null;
p := new ;
p* := 2
bool[3]  Y := [True, True, False] ;
int[2][2]  Z := [[0,1],[2,3]] ;
class estudiante antonio := estudiante(1, 2);
int (int , int ) potencia(a,b) {
  if b <0
    return 0;
  } elsif b == 0
    return 1;
  } else
    return potencia(a,b-1);
  }
};
```

Solo nos queda por comentar la función new que es una función sobrecargada que permite asignarle memoria dinámica a un puntero.

3.2. ;

Como en C++, el ; sirve para unir dos instrucciones en una sola.

3.3. While

El siguiente tipo de instrucciones son las formadas por un while. Se usa la palabra reservada `while` y la sintaxis es la habitual,

while (expresión booleana) { instrucción } y además, obligamos a colocar { y } para mostrar su ámbito.

3.4. For

El siguiente tipo de instrucciones son las formadas por un for. Se usa la palabra reservada `for` y la sintaxis es la habitual, aunque la variable contadora se declara fuera de este. Como antes obligamos a colocar { y } para describir su ámbito.

3.5. If, elsif y else

Las instrucciones if, elsif y else con la sintaxis habitual que usan palabras reservadas `if`, `elsif` y `else` respectivamente. Igual que en todos los casos anteriores, es obligatorio el uso de { } ‘para declarar el ámbito de cada uno de los bloques.

3.6. Switch

Vemos ahora el siguiente tipo de instrucción, las formadas por un switch. Usamos las palabras reservadas `switch`, `case` y `default`. No es necesario usar break ya que cada switch es independiente del otro pero sí es obligatorio colocar una cláusula default al final. La instrucción switch valora una expresión booleana una sola vez y busca entre cada uno de los casos el que se corresponda con esa evaluación.

3.7. Entrada y salida

Finalmente el último tipo de instrucción. Para la salida estándar está reservada la palabra `print` que imprime el contenido de una variable primitiva por la salida estándar. Para la entrada estándar, tenemos la palabra reservada `read` que de igual manera recoge el valor de una variable primitiva dada por la entrada estándar.

Finalmente, mostramos un ejemplo en el que se usan todas las instrucciones para ejemplificar su uso:

```
1  int X := 100;
2  int Y := 0;
3  int Z := 1;
4  int T ;
5  int(int, int) f(a,b) {
6    if 2*a < b
7      return 2*a + 1;
8    } elsif a > b
9      return b;
10   } else
11     return a + 1;
12   }
13 };
14 for (Y := 0; Y < X; Y := f(Y,X)) {
15   for (T := 0; T < Y; T := T + 1) {
16     if 2*Y < X
17       Z := Z + 1;
18     }
19   }
20 }
21 int Y := X - Z;
22 while (Y < X) {
23   read(T);
24   switch(T - 2*Y)
25     case 0 {
26       T := X;
27     } case 1 {
28       T := 2*X;
29     } case 2 {
30       T := 0;
31     } default {
32       T := Y;
33     }
34   }
35   Y := Y + 1;
36 }
37 print(Y);
```

Capítulo 4

Gestión de errores en C-Horizon

Para cada error se indicará su tipo y su posición en el archivo (fila y columna). La clase GestionErroresExp se encarga de actuar conforme a cada error. Además, llevará un contador de errores que imprimirá cuando haya habido algún error. Cuando no, permitirá la generación de código. Vamos a explicar los distintos tipos de errores.

4.1. Errores léxicos

Simplemente se imprimirá el token no reconocido.

Input [input6 (err lexico).txt][1]

```
1 int  hola ::= 2;
2 while  (True) {
3   hola++;_
4 }
5 int (int ) f(a) {
6   return a;
7 };
```

Output

- `[Fila 1, Columna 10] Caracter inesperado: ":"`
- `[Fila 3, Columna 9] Caracter inesperado: "_"`

[1]Cuando pongamos un input para mostrar la salida, indicaremos así el nombre del archivo de texto concreto para poder ejecutar el código. Estos archivos se encuentran en la carpeta tests.

4.2. Errores sintácticos

Si el error se encuentra en una instrucción simple, se retomará la compilación justo donde termina la instrucción, esto es, después de ";". En caso de que el error se dé en la declaración de un ámbito (una función, una condición, un bucle) se retomará la compilación justo al terminar ese ámbito. Es decir, se buscará el caracter "{" y se omitirá todo el código hasta encontrar el "}"que lo cierre (para ello también se llevará un contador de los "{"sin cerrar). Algunos ejemplos de errores sintácticos:

Input [input7 (err sintaxis).txt]

```
1 int  hola := 2;
2 while  (int ) {
3   hola++;
4 }
5 int (int ) f(a) {
6   return .;
7 };
```

Output

- [Fila 2, Columna 8] Elemento inesperado ‘‘int’’
- [Fila 6, Columna 9] Elemento inesperado ‘‘.’’

4.3. Errores de vinculación

Todos los nodos del árbol AST implementan dos funciones: bindFunc y bind. La primera, bindFunc, solo inserta una entrada en la tabla de símbolos actual con la declaración de una función. Esto lo hacemos para permitir que una función declarada en un bloque pueda ser usada en cualquier punto del bloque. Por lo tanto, cada vez que abramos un nuevo bloque, añadiremos una nueva tabla de símbolos para después llamar siempre primero a bindFunc y luego a bind del prgrama de ese bloque. Al terminar de vincular el bloque desapilaremos la tabla de símbolos. Además al desapilarla, guardaremos la tabla del bloque porque nos es útil luego (para la definición de structs y clases y para el tipado). Las tablas están implementadas como mapas de clave String y valor Declaracion. La pila de tablas es en realidad una lista, ya que resulta más eficiente.

Cuando lleguemos, usando bind, a una declaración de variable, insertaremos dicha declaración en el tabla de símbolos actual. Al insertar una declaración de variable comprobaremos que no se ha insertado otra declaración de variable con el mismo identificador. Sí se puede declarar una variable y una función con el mismo identificador. Como también existe sobrecarga, en vinculación permitimos que dos declaraciones de funciones tengan mismo identificador. Será en tipado donde contrastaremos si ambas tienen mismo tipo funcional (y lanzaremos un error de ambigüedad).

Al usar un identificador,se buscará en hacia atrás en la pila de tablas y se añadirán al conjunto de vínculos del identificador todas las declaraciones que encajen. Cuando en un bloque se encuentre una declaración de variable que coincida, no se buscará más declaraciones de variables en los bloques previos (ocultación). Sin embargo, sí se seguirán buscando declaraciones de funciones a las que poder vincular el identificador.

Por último, también llevaremos paralelamente otra pila de tablas que recoja los tipos nuevos (clases o structs) declarados en cada bloque. Cuando se defina un tipo nuevo, insertaremos en la tabla de tipos actual una entrada nueva. Esta tabla funciona exactamente igual que la anterior pero los pares son String (nombre del tipo nuevo) y NodoTipo (el tipo concreto).

Vamos a ver un ejemplo de un código que usa lo que acabamos de explicar y no da ningún error.

Input [input13 (err vinculacion6)]

```
1 int (int ) f (a) {};
2 bool  f;
3 {
4   int  f; //Declaración de variable "f"que no produce error por ocultación
5   int (bool ) f (a) {};
6   //Lo siguiente no produce error de declaración ni de tipado porque f se
    ha vinculado correctamente a la declaración del primer bloque
7   int  x := f(2);
8   //Lo siguiente tampoco proudcirá error de tipado porque f ha sido
    vinculada correctamente a la declaración de este bloque
9   x := f;
10 }
```

A continuación, veremos los tipos de errores concretos de vinculación.

4.3.1. Identificador ya declarado

Este error se lanza cuando se declara un identificador que ya ha sido declarado.

Input [input8 (err vinculacion1).txt]

```
1 int  hola;
2 bool  hola; //Error por identificador ya declarado
3 int (int ) hola (a) ;
4 int (int ) hola (a) ; //Resulta en un error de tipado, no de vinculación
```

Output

- `[Fila 2, Columna 6] Identificador ya declarado: hola`

4.3.2. Identificador no declarado

Este error se lanza cuando utiliza un identificador que no ha sido declarado previamente.

Input [input9 (err vinculacion2).txt]

```
1 int x;
2 x := a + 1; //Error de vinculación porque no se ha declarado "a"
3 x := f(a); //Solo hay error de id no declarado en a
4 {
5   int a;
6   x := a + 1; //Esta instrucción sí que es correcta
7 }
8 x := a + 1; //Ahora vuelve a dar error porque se ha cerrado el bloque
9 int (int ) f (num) ;
```

Output

- `[Fila 2, Columna 6] Identificador no declarado: a`
- `[Fila 3, Columna 8] Identificador no declarado: a`
- `[Fila 8, Columna 6] Identificador no declarado: a`

4.3.3. Tipo nuevo ya definido

Este error se lanza cuando se define un tipo nuevo con un nombre con el que ya se había definido un tipo nuevo antes. Aunque sean tipos nuevos distintos (una clase y un struct), si ambos tienen el mismo nombre lanzamos error porque puede producir ambigüedad en el futuro.

Input [input10 (err vinculacion3).txt]

```
1 class A {
2   int num;
3 };
4 struct A {}; //Error porque ya se ha definido un tipo nuevo con el nombre
  "A"
5 class A a; //Esta declaración es correcta, pues solo se acepta la primera
  definición de "A"
6 a.num := 2;
```

Output

- `[Fila 4, Columna 8] Tipo nuevo ya definido con ese nombre: A`

4.3.4. Tipo nuevo no definido

Este error se lanza cuando se declara una variable de un tipo nuevo que no ha sido declarado previamente.

Input [input11 (err vinculacion4).txt]

```
1 struct A a; //Error porque no se ha definido ese tipo nuevo
2 struct A {}; //Ahora sí se ha declarado, las declaraciones no darán error
3 struct A a1;
4 {
5   struct A a2;
6   struct B {};
7 }
8 struct B b; //Error porque no hay una declaración del struct B visible
```

Output

- `[Fila 1, Columna 1] Tipo nuevo no definido: A`
- `[Fila 8, Columna 1] Tipo nuevo no definido: B`

4.3.5. Error en el número de argumentos

Este error se lanza cuando se declara una función y el número de argmuentos no corresponde con el número de los tipos de argumentos de la función. Este error lo detectamos en vinculación porque cuando se vincula la cabecera de la función, se insertan declaraciones en la tabla de símbolos con cada uno de los argumentos, para que los identificadores del ámbito de la función se puedan vincular a estas declaraciones. Para que estas declaraciones no choquen con otras declaraciones de ese bloque, al vincular los parámetros de una función abrimos un nuevo bloque intermedio entre el bloque donde está definida la función y el bloque del ámbito de la función.

Input [input12 (err vinculacion5).txt]

```
1 int (int ) f (a,b) {};
```

Output

- `[Fila 1, Columna 10] Error en el número de argumentos de la función: f`